

10장. JavaScript 함수와 자료구조

학습 목표 - 함수 선언·표현식·화살표 함수를 상황에 맞게 작성할 수 있습니다 - 스코프와 클로저의 기본 개념을 설명할 수 있습니다 - 배열과 객체로 데이터를 구조화하고 조작할 수 있습니다 - map·filter·reduce 등 고차 메서드로 데이터를 선언적으로 처리할 수 있습니다

10.1 함수로 코드 재사용하기

재사용

복잡한 코드 -> 구조화(기능별 묶을 때)

도입

같은 계산을 여러 곳에서 반복해야 한다고 생각해 보십시오. 코드를 복사·붙여넣기 하면 당장은 동작하지만, 나중에 수정할 때 모든 복사본을 찾아서 고쳐야 합니다. **함수(Function)**는 이 문제를 해결합니다. 재료를 넣으면 정해진 음식을 만들어 내주는 요리 기계처럼, 함수는 입력을 받아 결과를 돌려주는 코드 묶음입니다.

핵심 개념 설명

선언문

표현식

함수 선언문(Function Declaration)

함수를 정의하는 가장 전통적인 방법입니다. `function` 키워드 뒤에 이름을 붙이고, 괄호 안에 매개변수(Parameter)를 나열합니다.

// 이름과 나이를 받아 인사 메시지를 반환하는 함수

```
function greet(name, age) {  
  const message = `안녕하세요, ${name}님! 만 ${age}세이시군요.`;  
  return message;  
}
```

```
const result = greet("김민준", 28);  
console.log(result);
```

// 콘솔 출력: 안녕하세요, 김민준님! 만 28세이시군요.

줄별 코드 설명

줄	코드	설명
2	<code>function greet(name, age) {</code>	<code>greet</code> 라는 이름의 함수를 선언합니다. <code>name</code> 과 <code>age</code> 는 매개변수입니다
3	<code>const message = \ ...`</code>	템플릿 리터럴로 인사 문자열을 조합하여 변수에 저장합니다
4	<code>return message;</code>	호출한 곳으로 값을 돌려줍니다. <code>return</code> 이후 코드는 실행되지 않습니다
7	<code>greet("김민준", 28)</code>	함수를 호출합니다. 전달하는 값을 인자(Argument) 라고 합니다

팁: 함수 선언문은 **호이스팅(Hoisting)**이 됩니다. 즉, 선언보다 위에서 호출해도 오류가 나지 않습니다. 하지만 코드를 위에서 아래로 읽을 수 있도록 선언 후 사용하는 습관을 권장합니다.

함수 표현식(Function Expression)

함수를 변수에 할당하는 방식입니다. 변수 이름이 곧 함수의 이름 역할을 합니다.

// 함수를 변수에 담는 함수 표현식

```
const square = function (n) {  
  return n * n;  
};
```

```
console.log(square(5));  
console.log(square(9));
```

// 콘솔 출력: 25

// 콘솔 출력: 81

줄별 코드 설명

줄	코드	설명
2	<code>const square = function (n) {</code>	이름이 없는 익명 함수를 만들어 <code>square</code> 변수에 할당합니다
3	<code>return n * n;</code>	매개변수 <code>n</code> 의 제공값을 반환합니다
6	<code>square(5)</code>	<code>square</code> 변수를 통해 함수를 호출합니다

주의: 함수 표현식은 호이스팅이 되지 않습니다. 선언 이전에 호출하면

`ReferenceError` 가 발생합니다.

화살표 함수(Arrow Function)

ES2015에서 도입된 더 짧은 함수 표기법입니다. `=>` 기호를 사용하며 콜백 함수로 자주 쓰입니다.

// 화살표 함수의 다양한 형태

const add = (a, b) $\Rightarrow$ a + b;

const double = n $\Rightarrow$ n * 2;

const sayHello = () $\Rightarrow$ console.log("안녕하세요!"); // 매개변수 없으면 빈 괄호 필수

console.log(add(3, 7));

console.log(**double**(6));

sayHello();

// 식이 한 줄이면 중괄호와 **return**

// 매개변수가 하나면 괄호 생략

// 콘솔 출력: 10

// 콘솔 출력: 12

// 콘솔 출력: 안녕하세요!

줄별 코드 설명

줄	코드	설명
2	<code>(a, b) $\Rightarrow$ a + b</code>	중괄호와 <code>return</code> 없이 표현식 결과를 바로 반환합니다
3	<code>n $\Rightarrow$ n * 2</code>	매개변수가 하나일 때 괄호를 생략할 수 있습니다
4	<code>() $\Rightarrow$ console.log(...)</code>	매개변수가 없을 때는 빈 괄호 <code>()</code> 를 반드시 씁니다

매개변수 심화: 기본값과 나머지 매개변수

기본 매개변수(Default Parameter)는 인자가 전달되지 않았을 때 사용할 값을 지정합니다. 나머지 매개변수(Rest Parameter)는 여러 인자를 배열로 모아 받습니다.

// 기본값 매개변수: 단위 미지정 시 'km'를 기본값으로 사용

```
function formatDistance(value, unit = "km") {  
  return `${value}${unit}`;  
}
```

// 나머지 매개변수: 개수에 관계없이 숫자들을 합산

```
const sumAll = (...numbers) => {  
  return numbers.reduce((total, n) => total + n, 0);  
};
```

```
console.log(formatDistance(42));  
console.log(formatDistance(1500, "m"));  
console.log(sumAll(1, 2, 3, 4, 5));
```

```
// 콘솔 출력: 42km  
// 콘솔 출력: 1500m  
// 콘솔 출력: 15
```

줄별 코드 설명

줄	코드	설명
2	<code>unit = "km"</code>	<code>unit</code> 에 값을 전달하지 않으면 <code>"km"</code> 가 기본값으로 사용됩니다
7	<code>(...numbers)</code>	<code>...</code> 앞에 붙으면 나머지 매개변수입니다. 여러 인자를 배열로 받습니다
8	<code>numbers.reduce(...)</code>	받은 배열을 <code>reduce</code> 로 모두 더합니다(reduce는 10.5절에서 자세히 다룹니다)

함수 선언문 vs 표현식 vs 화살표 함수 비교

구분	호이스팅	this 바인딩	주 사용처
함수 선언문	O (전체)	호출 방식에 따름	모듈의 주요 함수
함수 표현식	X } 이름이 없어서	호출 방식에 따름	조건부 함수 정의
화살표 함수	X }	상위 스코프 상속	콜백, 짧은 변환 함수

절 요약

- 함수는 재사용 가능한 코드 묶음으로, 함수 선언문·표현식·화살표 함수 세 가지 방식으로 작성합니다
- 화살표 함수는 한 줄 표현식에서 종괄호와 `return` 을 생략할 수 있어 간결합니다
- 기본 매개변수로 인자 누락 시 기본값을 지정하고, 나머지 매개변수로 가변 인자를 배열로 받습니다

10.2 스코프와 클로저 맛보기

변수의 접근 가능한 유효 범위

Global Scope: 바깥 영역에 선언된 변수

Function Scope: 함수 내부에서 선언된 변수 -> var로 선언된 변수는 함수 내에서만 유효

Block Scope: 블록({ }) 내부에서 선언된 변수 -> let, const 변수는 블록 내부에서만 유효

도입

코드를 작성하다 보면 변수가 어디서 어디까지 보이는지 헷갈릴 때가 있습니다. **스코프 (Scope)**는 변수가 유효한 범위를 의미합니다. 그리고 **클로저(Closure)**는 함수가 자신이 만들어진 환경을 기억하는 특성으로, JavaScript의 핵심 동작 원리 중 하나입니다.

핵심 개념 설명

전역 스코프와 지역 스코프

함수 바깥에 선언된 변수는 **전역 스코프(Global Scope)**에 속하며 어디서든 접근할 수 있습니다. 함수 안에 선언된 변수는 **지역 스코프(Local Scope)**에 속하며 함수 밖에서는 접근할 수 없습니다.

```

const globalName = "전역 변수"; // 전역 스코프

function showScope() {
  const localName = "지역 변수"; // 지역 스코프 (함수 안에서만 유효)
  console.log(globalName); // 전역 변수 접근 가능
  console.log(localName); // 지역 변수 접근 가능
}

showScope();
console.log(globalName); // 전역 변수 접근 가능
// console.log(localName); // ReferenceError: localName is not defined

```

```

// 콘솔 출력: 전역 변수
// 콘솔 출력: 지역 변수
// 콘솔 출력: 전역 변수

```

줄별 코드 설명

줄	코드	설명
1	<code>const globalName = "전역 변수"</code>	함수 바깥에 선언되어 파일 어디서든 접근 가능합니다
4	<code>const localName = "지역 변수"</code>	<code>showScope</code> 함수 안에서만 유효한 변수입니다
10	<code>// console.log(localName)</code>	함수 밖에서는 <code>localName</code> 에 접근할 수 없어 오류가 납니다

블록 스코프(Block Scope)

`const` 와 `let` 은 `{ }` 블록 단위 스코프를 가집니다. `if` , `for` , `while` 의 중괄호도 스코프 경계가 됩니다.

```

if (true) {
  const blockVar = "블록 안 변수"; > {} 안에서 선언
  let blockLet = "블록 안 let";
  console.log(blockVar); // 정상 접근
}
// console.log(blockVar); // ReferenceError: blockVar is not defined
// console.log(blockLet); // ReferenceError: blockLet is not defined

for (let i = 0; i < 3; i++) {
  console.log(i); // 0, 1, 2
}
// console.log(i); // ReferenceError: i is not defined

```

```

// 콘솔 출력: 블록 안 변수
// 콘솔 출력: 0
// 콘솔 출력: 1
// 콘솔 출력: 2

```

스코프 체인(Scope Chain)

내부 함수는 자신의 스코프에서 변수를 찾고, 없으면 바깥 스코프로 올라가며 찾습니다. 이 연쇄 탐색 과정을 **스코프 체인**이라고 합니다.


```
const outer = "바깥 변수";
```

```
선언 function outerFn() {  
  const middle = "중간 변수";
```

```
선언 function innerFn() {  
  const inner = "안쪽 변수";  
  // 안쪽 → 중간 → 전역 순으로 변수를 찾습니다  
  console.log(inner); // 자신의 스코프에서 발견 내부 탐색(발견)  
  console.log(middle); // 바깥 함수 스코프에서 발견 내부 탐색(x) -> outerFn탐색(발견)  
  console.log(outer); // 전역 스코프에서 발견 내부 탐색(x) -> outerFn탐색(x)->전역탐색(발견)  
}
```

```
호출 innerFn();  
}
```

```
호출 outerFn();
```

↑
스cope 체인

```
// 콘솔 출력: 안쪽 변수  
// 콘솔 출력: 중간 변수  
// 콘솔 출력: 바깥 변수
```

줄별 코드 설명

줄	코드	설명
6	<code>function innerFn() {</code>	<code>outerFn</code> 안에 정의된 중첩 함수입니다
9	<code>console.log(middle)</code>	<code>innerFn</code> 의 스코프에 없으므로 한 단계 위인 <code>outerFn</code> 스코프에서 찾습니다
10	<code>console.log(outer)</code>	함수 스코프에 없으므로 전역 스코프에서 찾습니다

렉시컬 스코프: 함수를 어디서 호출했는지가 아니라
어디서 정의(선언)했는가에 따라 상위 스코프 결정

클로저(Closure)의 개념과 활용

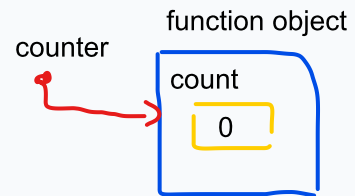
정적 스코프

클로저는 함수가 선언될 때의 외부 변수(렉시컬 ~~환경~~ ^{스cope})를 기억하여, 그 함수가 바깥에서 호출되
어도 해당 변수에 계속 접근할 수 있는 특성입니다. 도시락을 싸 온 사람이 밖에 나가서도 집에
서 담아 온 반찬을 꺼내 먹을 수 있는 것과 같습니다.

아래 예제는 클로저를 이용해 카운터를 만듭니다. 내부 함수가 바깥 변수 `count` 를 기억한 채로 반환됩니다.

A 상태 보존
 // 클로저로 카운터 만들기
function makeCounter() {
 let count = 0; // 이 변수는 반환된 함수가 기억합니다

return function () {
 count += 1; count = count + 1;
 console.log(`현재 카운트: \${count}`);
 };
}



함수를 실행
const counter = makeCounter(); // makeCounter 실행 후 내부 함수만 반환
counter(); // count를 기억한 채 실행
counter();
counter();
 counter에 return function() {}이 들어감.
 count에 접근은 안 됨.

이전 상태를 보호하기 위해 클로저 사용
 비동기 처리 시 이전 상태를 기억하고 있어야 함.

// 콘솔 출력: 현재 카운트: 1
 // 콘솔 출력: 현재 카운트: 2
 // 콘솔 출력: 현재 카운트: 3

줄별 코드 설명

줄	코드	설명
2	<code>let count = 0</code>	<code>makeCounter</code> 스코프의 변수. 반환된 함수가 이 변수를 클로저로 기억합니다
4	<code>return function () {</code>	익명 함수를 반환합니다. 이 함수가 <code>count</code> 를 참조하여 클로저가 만들어집니다
10	<code>const counter = makeCounter()</code>	<code>makeCounter</code> 는 실행되어 사라지지만, 반환된 함수는 <code>count</code> 를 살아있게 합니다
11	<code>counter()</code>	호출할 때마다 <code>count</code> 가 누적됩니다

팁: 클로저는 어렵고 특수한 기능처럼 느껴지지만, 함수가 외부 변수를 참조하면 자연스럽게 발생합니다. 앞선 `makeCounter` 예제처럼 상태를 안전하게 캡슐화할 때 매우 유용합니다.

베스트 프랙티스: 클로저로 만든 변수는 외부에서 직접 수정할 수 없습니다. 이 특성을 이용해 전역 변수 오염 없이 상태를 관리하는 패턴이 실무에서 자주 쓰입니다.

절 요약

- 변수는 선언된 위치에 따라 전역·지역·블록 스코프를 가집니다
- 내부 함수는 스코프 체인을 통해 바깥 스코프의 변수에 접근합니다
- 클로저는 함수가 선언 당시의 렉시컬 환경을 기억하는 특성으로, 상태 캡슐화에 활용됩니다

10.3 배열 다루기

도입

여러 데이터를 하나로 묶어 관리해야 할 때 **배열(Array)**을 사용합니다. 번호가 매겨진 사물함 한 줄에 물건을 차례로 넣어 두는 것과 같습니다. JavaScript의 배열은 다양한 내장 메서드를 제공하여 데이터 조작을 편리하게 합니다.

핵심 개념 설명

배열 생성과 인덱스 접근

배열은 **대괄호 `[]`**로 만들고, **`0`부터 시작하는 인덱스(Index)**로 각 요소에 접근합니다.

// 배열 생성과 기본 접근

```
const fruits = ["사과", "바나나", "딸기", "포도"];

console.log(fruits[0]);           // 첫 번째 요소
console.log(fruits[2]);           // 세 번째 요소
console.log(fruits.length);       // 배열 길이
console.log(fruits[fruits.length - 1]); // 마지막 요소
```

```
// 콘솔 출력: 사과
// 콘솔 출력: 딸기
// 콘솔 출력: 4
// 콘솔 출력: 포도
```

줄별 코드 설명

줄	코드	설명
2	<code>const fruits = [...]</code>	배열 리터럴로 생성합니다. <code>const</code> 이지만 배열의 내용은 변경할 수 있습니다
4	<code>fruits[0]</code>	인덱스는 0부터 시작합니다. 첫 번째 요소는 <code>fruits[0]</code> 입니다
6	<code>fruits.length</code>	배열에 담긴 요소의 개수를 반환합니다
7	<code>fruits[fruits.length - 1]</code>	<code>length - 1</code> 이 마지막 인덱스입니다

`Array.prototype.length`

항목	내용
설명	배열에 담긴 요소의 개수를 반환합니다
타입	<code>number</code> (읽기/쓰기 가능)
사용 예시	<code>[1, 2, 3].length</code> → 3

요소 추가·제거 메서드

```
const colors = ["빨강", "파랑"];

// 끝에 추가 / 제거
colors.push("초록");           // 끝에 "초록" 추가
const last = colors.pop();     // 끝 요소("초록") 제거 후 반환

// 앞에 추가 / 제거
colors.unshift("노랑");        // 앞에 "노랑" 추가
const first = colors.shift();  // 앞 요소("노랑") 제거 후 반환

console.log(colors);
console.log(`제거된 마지막: ${last}, 제거된 첫 번째: ${first}`);
```

```
// 콘솔 출력: ["빨강", "파랑"]
// 콘솔 출력: 제거된 마지막: 초록, 제거된 첫 번째: 노랑
```

줄별 코드 설명

줄	코드	설명
4	<code>colors.push("초록")</code>	배열 끝에 요소를 추가합니다. 반환값은 추가 후 배열 길이입니다
5	<code>colors.pop()</code>	배열 끝 요소를 제거하고 그 값을 반환합니다
8	<code>colors.unshift("노랑")</code>	배열 앞에 요소를 추가합니다
9	<code>colors.shift()</code>	배열 첫 번째 요소를 제거하고 그 값을 반환합니다

`Array.prototype.push( ... items )`

항목	내용
설명	배열 끝에 하나 이상의 요소를 추가하고, 추가 후 배열의 <code>length</code> 를 반환합니다
매개변수	<code>items</code> (any, 필수): 추가할 요소들
반환값	<code>number</code> - 추가 후 배열의 길이
사용 예시	<code>[1, 2].push(3) → 3</code> (배열은 <code>[1, 2, 3]</code> 으로 변경)

slice와 splice의 차이

두 메서드는 이름이 비슷하지만 동작이 다릅니다. `slice` 는 원본을 건드리지 않고 복사하고, `splice` 는 원본을 직접 수정합니다.

```
const numbers = [10, 20, 30, 40, 50];

// slice: 원본 유지, 지정 구간을 새 배열로 반환
const sliced = numbers.slice(1, 4); // 인덱스 1~3 (4 미포함)
console.log(sliced);                // [20, 30, 40]
console.log(numbers);               // 원본 변경 없음

// splice: 원본 수정, 지정 위치에서 요소를 제거하거나 삽입
const removed = numbers.splice(1, 2); // 인덱스 1부터 2개 제거
console.log(removed);                 // [20, 30] (제거된 요소)
console.log(numbers);                 // [10, 40, 50] (원본 변경됨)
```

```
// 콘솔 출력: [20, 30, 40]
// 콘솔 출력: [10, 20, 30, 40, 50]
// 콘솔 출력: [20, 30]
// 콘솔 출력: [10, 40, 50]
```

줄별 코드 설명

줄	코드	설명
4	<code>numbers.slice(1, 4)</code>	인덱스 1 이상 4 미만(1, 2, 3) 구간을 새 배열로 복사합니다. 원본 불변
9	<code>numbers.splice(1, 2)</code>	인덱스 1부터 2개 요소를 제거합니다. 원본을 직접 변경합니다

Array.prototype.slice(start?, end?)

항목	내용
설명	배열의 일부를 복사한 새 배열을 반환합니다. 원본을 변경하지 않습니다
매개변수	<code>start</code> (number, 선택): 시작 인덱스. 음수면 끝에서부터 셉니다. 기본값 <code>0</code>
	<code>end</code> (number, 선택): 종료 인덱스(미포함). 기본값 <code>array.length</code>
반환값	<code>Array</code> - 복사된 새 배열
사용 예시	<code>[1,2,3,4].slice(1,3) → [2, 3]</code>

배열 순회하기

```

      0       1       2
const animals = ["강아지", "고양이", "토끼"];

// for...of로 순회 (값만 필요할 때 권장)
for (const animal of animals) {
  console.log(animal);
}

// forEach로 인덱스와 함께 순회
animals.forEach((animal, index) => {
  console.log(`${index + 1}번째: ${animal}`);
});

```

✖ 스코프 : 변수의 유효한 범위 -> 전역, 지역(함수, 블록 안)
 스코프체인 : 지역 찾고 없으면 전역에서 찾기
 클로저 : 함수 선언될 당시 외부 환경 기억
 ↳ 비공개 상태 안전하게 관리

```

// 콘솔 출력: 강아지
// 콘솔 출력: 고양이
// 콘솔 출력: 토끼
// 콘솔 출력: 1번째: 강아지
// 콘솔 출력: 2번째: 고양이
// 콘솔 출력: 3번째: 토끼

```

줄별 코드 설명

줄	코드	설명
4	<code>for (const animal of animals)</code>	배열 요소를 순서대로 하나씩 꺼내 <code>animal</code> 에 할당합니다
9	<code>animals.forEach((animal, index...</code>	콜백 함수의 첫 인자는 요소 값, 두 번째는 인덱스입니다

주의: `push`, `pop`, `splice` 는 원본 배열을 변경(mutation)합니다. 원본을 보존해야 할 때는 `slice` 나 스프레드 연산자(`[...arr]`)로 복사본을 만들어 사용하십시오.

절 요약

- 배열은 `[]` 로 생성하며, `0` 부터 시작하는 인덱스로 요소에 접근합니다
- `push/pop` 은 끝, `unshift/shift` 는 앞의 요소를 추가·제거합니다
- `slice` 는 원본을 보존하며 복사하고, `splice` 는 원본을 직접 수정합니다

변수들을 하나로 묶어 관리

10.4 객체로 데이터 구조화하기

object 변수 : 저장(관련 내용 하나로 묶어 저장)
 메소드 : 처리

도입

이름·나이·직업처럼 관련된 여러 정보를 하나로 묶고 싶을 때 **객체(Object)**를 사용합니다. 이름·나이·주소 항목이 적힌 신분증처럼, 객체는 관련 정보를 하나의 단위로 묶어 의미 있게 다룹니다.

리터럴

핵심 개념 설명

대입되는 값

변수 = 대입값

객체 리터럴로 데이터 묶기

중괄호 `{ }` 로 객체를 만들고, **키(Key): 값(Value)** 쌍으로 **속성(Property)**을 정의합니다.

// 사람 정보를 담은 객체

```
const person = {  
  name: "이수아",  
  age: 32,  
  city: "서울",  
  isStudent: false,  
};
```

```
console.log(person.name);
```

// 점 표기법

```
console.log(person["age"]);
```

// 대괄호 표기법

```
console.log(person.city);
```

// 콘솔 출력: 이수아

// 콘솔 출력: 32

// 콘솔 출력: 서울

줄별 코드 설명

줄	코드	설명
2~7	<code>const person = { ... }</code>	객체 리터럴로 여러 속성을 한번에 정의합니다
9	<code>person.name</code>	점 표기법. 속성 이름이 정해진 경우에 사용합니다
10	<code>person["age"]</code>	대괄호 표기법. 속성 이름이 변수에 담겨 있거나 동적일 때 사용합니다

속성 추가·삭제

(변수)

object 안에서

객체는 선언 후에도 속성을 자유롭게 추가하거나 삭제할 수 있습니다.

object 생성

오브젝트 생성

```
const car = { brand: "현대", model: "아이오닉" };

// 속성 추가
car.color = "흰색";
car.year = 2024;

// 속성 삭제
delete car.model;

console.log(car);
console.log("model" in car); // in 연산자로 속성 존재 확인
```



```
// 콘솔 출력: {brand: "현대", color: "흰색", year: 2024}
// 콘솔 출력: false
```

줄별 코드 설명

줄	코드	설명
4	<code>car.color = "흰색"</code>	존재하지 않는 키에 값을 할당하면 새 속성이 추가됩니다
7	<code>delete car.model</code>	<code>delete</code> 연산자로 속성을 완전히 제거합니다
10	<code>"model" in car</code>	<code>in</code> 연산자는 해당 속성이 객체에 있으면 <code>true</code> 를 반환합니다

객체 안의 메서드

값이 함수인 속성을 **메서드(Method)**라고 합니다. 객체와 관련된 동작을 함께 캡슐화합니다.

속성 : 값
key : value

예.
`sum : function() { 코드 }`

객체 안에 있어서 메소드, 객체 밖에 있으면 함수.

calculator



```
const calculator = {
  result: 0,

  add(n) {                // 메서드 단축 표기
    this.result += n;
    return this;          // 체이닝을 위해 자기 자신을 반환
                           // 연결
  },

  subtract(n) {
    this.result -= n;
    return this;
  },

  getValue() {
    return this.result;
  },
};
```

```
calculator.add(10).add(5).subtract(3);    return에 this 반환되어서 계속 추가 가능.
console.log(calculator.getValue());
```

```
// 콘솔 출력: 12
```

줄별 코드 설명

줄	코드	설명
4	<code>add(n) {</code>	<code>add: function(n) {}</code> 의 단축 표기입니다
5	<code>this.result += n</code>	<code>this</code> 는 메서드를 호출한 객체 자신, 즉 <code>calculator</code> 를 가리킵니다
6	<code>return this</code>	자기 자신을 반환하면 메서드를 연속으로 호출(체이닝)할 수 있습니다

주의: 화살표 함수로 메서드를 정의하면 `this`가 상위 스코프의 `this`를 가리키므로, 객체 메서드에서는 일반 함수 표기(`메서드명() {}`)를 사용하십시오.

구조 분해 할당(Destructuring Assignment)과 단축 표기

구조 분해 할당은 객체의 속성 값을 손쉽게 개별 변수로 꺼내는 문법입니다.

```
const product = {
  id: "P-001",
  name: "무선 키보드",
  price: 89000,
  stock: 15,
};

// 구조 분해 할당: 속성을 개별 변수로 추출
const { name, price, stock } = product;
console.log(`${name}: ${price.toLocaleString()}원 (재고 ${stock}개)`);

// 다른 변수명으로 받기
const { name: productName, price: productPrice } = product;
console.log(`상품명: ${productName}, 가격: ${productPrice}`);

// 기본값 설정
const { discount = 0, name: pName } = product;
console.log(`할인율: ${discount}%, 상품: ${pName}`);
```

```
// 콘솔 출력: 무선 키보드: 89,000원 (재고 15개)
// 콘솔 출력: 상품명: 무선 키보드, 가격: 89000
// 콘솔 출력: 할인율: 0%, 상품: 무선 키보드
```

줄별 코드 설명

줄	코드	설명
9	<code>const { name, price, stock } = ...</code>	객체에서 동일한 이름의 변수를 꺼냅니다
14	<code>const { name: productName ... }</code>	<code>name</code> 속성을 <code>productName</code> 이라는 변수명으로 받습니다
18	<code>discount = 0</code>	<code>discount</code> 속성이 없으면 기본값 <code>0</code> 을 사용합니다

객체와 배열을 함께 활용하기

실제 데이터는 종종 객체의 배열 형태로 다룹니다.

```
const students = [
  { name: "김민지", score: 85 },
  { name: "이도현", score: 92 },
  { name: "박소연", score: 78 },
];

// 각 학생 정보 출력
students.forEach(({ name, score }) => {
  console.log(`${name}: ${score}점`);
});
```

```
// 콘솔 출력: 김민지: 85점
// 콘솔 출력: 이도현: 92점
// 콘솔 출력: 박소연: 78점
```

줄별 코드 설명

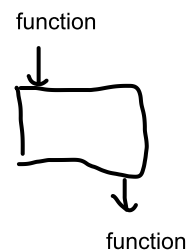
줄	코드	설명
1~5	<code>const students = [...]</code>	객체들의 배열입니다. 실무에서 가장 자주 보는 데이터 구조입니다
8	<code>forEach(({ name, score }) => {</code>	콜백 매개변수에서 바로 구조 분해 할당을 사용합니다

절 요약

- 객체는 키-값 쌍으로 관련 데이터를 묶는 자료구조입니다
- 점 표기법 또는 대괄호 표기법으로 속성에 접근합니다
- 구조 분해 할당으로 객체의 속성을 손쉽게 개별 변수로 꺼낼 수 있습니다
- 객체의 배열 형태는 실무에서 가장 흔한 데이터 구조입니다



10.5 고차 배열 메서드: map, filter, reduce



도입

배열을 처리할 때 `for` 문을 직접 쓸 수도 있지만, JavaScript는 더 읽기 쉬운 방법을 제공합니다. **고차 함수(Higher-order Function)**는 **다른 함수를 인자로 받거나 반환하는** 함수입니다. `map`, `filter`, `reduce` 는 배열을 선언적(declarative)으로 처리하는 대표적인 고차 메서드입니다. **공장 컨베이어에서 가공(map)·선별(filter)·합산(reduce) 공정을 차례로 거치는 것과 같습니다.**

체이닝으로 가능

`map`: 배열의 모든 요소를 콜백 함수에 따라 변환하여 새로운 배열로 반환.(체이닝 가능)

`filter`: 배열 각각의 요소에서 조건을 만족하는 요소만 골라내어 새 배열로 반환

map-filter-reduce 데이터 파이프라인

`reduce`: 배열을 순회하며 하나의 최종 결과값(합, 평균 등)으로 누적

핵심 개념 설명

콜백 함수(Callback Function)

콜백 함수는 다른 함수에 인자로 전달되어, 그 함수가 적절한 시점에 호출하는 함수입니다.

`map`, `filter`, `reduce` 는 모두 콜백 함수를 받아 배열의 각 요소에 적용합니다.

실질적으로는 콜백함수가 가공하는 것.
`map`, `filter`, `reduce`는 연결

`map`, `filter`, `reduce`는 고차 배열 메소드

// 콜백 함수의 기본 개념

```
const numbers = [1, 2, 3, 4, 5];
```

// 콜백 함수를 변수에 담아 전달

```
const isEven = (n) => n % 2 === 0;
```

콜백함수만 바꾸면 기능이 변함
=== 1 하면 홀수일 때 True 반환

callback은 true or false로 반환되어야

```
const evens = numbers.filter(isEven); // isEven이 콜백 함수
console.log(evens);
```

true or false 골라내는 함수

true인 값들이 배열로 반환되니
`evens = [2, 4]`

// 콘솔 출력: [2, 4]

map으로 변환하기

가공(적용해서 새 배열 반환)

`map` 은 배열의 모든 요소에 콜백 함수를 적용하여 **변환된 새 배열을 반환**합니다. 원본 배열은 변경되지 않습니다.

// 상품 가격 배열에 10% 할인 적용

```
const prices = [10000, 25000, 8000, 42000];
```

```
const discounted = prices.map((price) => price * 0.9); 90% 가격으로 가공  
console.log(discounted);
```

// 객체 배열 변환: 이름만 추출하기

```
const users = [  
  { id: 1, name: "홍길동", email: "hong@example.com" }, json  
  { id: 2, name: "김영희", email: "kim@example.com" },  
  { id: 3, name: "이철수", email: "lee@example.com" },  
];
```

```
const names = users.map((user) => user.name); 이름만 추출하여 이름만 들어간 배열로 가공  
console.log(names);
```

// HTML 문자열로 변환

```
const listItems = names.map((name) => `- ${name}</li>`);  
console.log(listItems.join("\n"));

```

```
// 콘솔 출력: [9000, 22500, 7200, 37800]  
// 콘솔 출력: ["홍길동", "김영희", "이철수"]  
// 콘솔 출력: <li>홍길동</li>  
//           <li>김영희</li>  
//           <li>이철수</li>
```

줄별 코드 설명

줄	코드	설명
4	<code>prices.map((price) => price * ...</code>	각 가격에 0.9를 곱한 새 배열을 반환합니다. <code>prices</code> 는 변경되지 않습니다
14	<code>users.map((user) => user.name)</code>	객체 배열에서 특정 속성만 추출하는 패턴입니다
18	<code>names.map((name) => \</code> <code>...`)</code>	각 이름을 HTML 문자열로 변환합니다
19	<code>.join("\n")</code>	배열 요소를 줄바꿈으로 연결한 문자열로 만듭니다

Array.prototype.map(callbackFn)

항목	내용
설명	배열의 각 요소에 콜백 함수를 적용한 결과로 이루어진 새 배열을 반환합니다
매개변수	<code>callbackFn(element, index, array)</code> : 요소마다 호출되는 함수. 반환값이 새 배열의 요소가 됩니다
반환값	<code>Array</code> - 변환된 새 배열 (원본 불변)
사용 예시	<code>[1, 2, 3].map(n => n * 2) → [2, 4, 6]</code>

filter로 걸러내기

`filter` 는 콜백 함수가 `true` 를 반환하는 요소만 모아 새 배열을 반환합니다.

```
const products = [
  { name: "노트북", price: 1200000, inStock: true },
  { name: "마우스", price: 35000, inStock: false },
  { name: "키보드", price: 89000, inStock: true },
  { name: "모니터", price: 450000, inStock: true },
  { name: "헤드셋", price: 120000, inStock: false },
];

// 재고 있는 상품만 필터링
const available = products.filter((product) => product.inStock);
console.log(available.map((p) => p.name));

// 10만 원 이하 재고 상품
const affordable = products.filter(
  (product) => product.inStock && product.price <= 100000
);
console.log(affordable.map((p) => `${p.name}: ${p.price.toLocaleString()}원`));
```

products.filter(함수).map((p) => p.name)
재고 있는 상품 이름 반환
배열로 반환

우리나라 화폐 단위

```
// 콘솔 출력: ["노트북", "키보드", "모니터"]
// 콘솔 출력: ["키보드: 89,000원"]
```


줄별 코드 설명

줄	코드	설명
10	<code>products.filter((product) => p...</code>	<code>inStock</code> 이 <code>true</code> 인 요소만 추려 새 배열을 만듭니다
13~15	<code>filter((product) => ... && ...)</code>	여러 조건을 <code>&&</code> 로 결합할 수 있습니다
16	<code>.map((p) => ...)</code>	<code>filter</code> 결과에 바로 <code>map</code> 을 체이닝합니다

Array.prototype.filter(callbackFn)

항목	내용
설명	콜백 함수가 <code>true</code> 를 반환하는 요소만 포함한 새 배열을 반환합니다
매개변수	<code>callbackFn(element, index, array)</code> : 요소마다 호출되는 판별 함수. <code>true</code> 이면 포함
반환값	<code>Array</code> - 필터링된 새 배열 (원본 불변)
사용 예시	<code>[1, 2, 3, 4].filter(n => n > 2) → [3, 4]</code>

reduce로 누적 계산하기

`reduce` 는 배열을 순회하며 **하나의 결과 값으로 누적**합니다. 합계, 평균, 객체 변환 등 다양하게 활용됩니다.

```

const orders = [
  { item: "아메리카노", qty: 2, unitPrice: 4500 },
  { item: "카페라떼", qty: 1, unitPrice: 5500 },
  { item: "샌드위치", qty: 3, unitPrice: 7000 },
];

// 총 주문 금액 계산 (누적기 초기값: 0) 누적값      현재 요소
const totalAmount = orders.reduce((accumulator, order) => {
  return accumulator + order.qty * order.unitPrice;  return 값이 다음 반복의 accumulator가 됨.
}, 0);
// 초기 값      수량 * 가격 = 총 가격

console.log(`총 금액: ${totalAmount.toLocaleString()}원`);

// 최댓값 구하기
const scores = [72, 91, 58, 84, 63, 97, 45];
const maxScore = scores.reduce((max, score) => (score > max ? score : max));
console.log(`최고 점수: ${maxScore}`);

```

```

// 콘솔 출력: 총 금액: 39,500원
// 콘솔 출력: 최고 점수: 97

```

줄별 코드 설명

줄	코드	설명
8	<code>reduce((accumulator, order) =>...</code>	<code>accumulator</code> 는 누적값, <code>order</code> 는 현재 요소입니다
9	<code>accumulator + order.qty * orde...</code>	누적값에 현재 주문 금액을 더합니다. 이 반환 값이 다음 호출의 <code>accumulator</code> 가 됩니다
10	<code>}, 0)</code>	<code>0</code> 은 <code>accumulator</code> 의 초기값입니다
14	<code>reduce((max, score) => ...)</code>	초기값을 생략하면 첫 번째 요소가 초기 <code>accumulator</code> 가 됩니다

Array.prototype.reduce(callbackFn, initialValue?)

항목	내용
설명	배열의 각 요소에 대해 콜백 함수를 실행하고, 하나의 누적 결과 값을 반환합니다
매개변수	<code>callbackFn(accumulator, currentValue, currentIndex, array)</code> : 요소마다 호출되는 누적 함수
	<code>initialValue</code> (any, 선택): 첫 번째 <code>accumulator</code> 초기값. 생략 시 배열의 첫 요소가 됩니다
반환값	누적 계산의 최종 결과값
사용 예시	<code>[1, 2, 3, 4].reduce((sum, n) => sum + n, 0) → 10</code>

메서드 체이닝(Method Chaining)

`map`, `filter`, `reduce` 는 연속으로 이어 쓸 수 있습니다. 각 메서드가 새 배열을 반환하기 때문입니다.

```
const employees = [
  { name: "박지훈", department: "개발", salary: 4800000 },
  { name: "최수민", department: "디자인", salary: 3900000 },
  { name: "김태양", department: "개발", salary: 5200000 },
  { name: "이나라", department: "디자인", salary: 4100000 },
  { name: "정우성", department: "개발", salary: 4500000 },
];

// 개발팀 직원들의 평균 월급 계산
const devSalaries = employees
  .filter((emp) => emp.department === "개발")
  .map((emp) => emp.salary);

const average = devSalaries.reduce((sum, s) => sum + s, 0) / devSalaries.length;

console.log(`개발팀 인원: ${devSalaries.length}명`);
console.log(`개발팀 평균 월급: ${Math.round(average).toLocaleString()}원`);
```

```
// 콘솔 출력: 개발팀 인원: 3명
// 콘솔 출력: 개발팀 평균 월급: 4,833,333원
```

줄별 코드 설명

줄	코드	설명
10	<code>.filter((emp) => emp.departmen...</code>	개발팀 직원만 추립니다. 새 배열을 반환하므로 바로 <code>.map()</code> 을 이어 쓸 수 있습니다
11	<code>.map((emp) => emp.salary)</code>	직원 객체 배열에서 급여 숫자 배열로 변환합니다
13	<code>reduce(..., 0) / devSalaries.l...</code>	합계를 인원수로 나눠 평균을 구합니다

forEach vs map 비교

```
const nums = [1, 2, 3];
// 하나씩 꺼내서 반복
// forEach: 반환값 없음. 부수 효과(콘솔 출력, DOM 조작 등)를 위해 사용
nums.forEach((n) => console.log(n * 10));

// map: 새 배열 반환. 변환 결과가 필요할 때 사용
const doubled = nums.map((n) => n * 10);
console.log(doubled);
```

```
// 콘솔 출력: 10
// 콘솔 출력: 20
// 콘솔 출력: 30
// 콘솔 출력: [10, 20, 30]
```

베스트 프랙티스: 변환된 결과가 필요하다면 `map` 을, 결과 없이 각 요소에 작업만 하면 `forEach` 를 사용하십시오. `map` 의 반환값을 사용하지 않는 패턴은 의도를 불명확하

게 합니다.

비교 테이블: map, filter, reduce, forEach

메서드	반환값	원본 변경	주요 용도
<code>map</code>	새 배열	X	각 요소 변환
<code>filter</code>	새 배열(일부)	X	조건에 맞는 요소 추출
<code>reduce</code>	단일 값 (모든 타입 가능)	X	누적 계산, 합산, 집계
<code>forEach</code>	<code>undefined</code>	X (콜백에서 원본 수정 가능)	순회 후 부수 효과 실행

절 요약

- 고차 함수는 콜백 함수를 인자로 받아 배열을 선언적으로 처리합니다
- `map` 은 변환, `filter` 는 선별, `reduce` 는 누적 계산을 담당합니다
- 세 메서드를 체이닝하면 복잡한 데이터 처리를 한눈에 읽히는 코드로 작성할 수 있습니다

FAQ

Q1. 화살표 함수와 일반 함수는 언제 무엇을 써야 하나요?

A1. 짧은 변환 로직이나 콜백 함수로 쓸 때는 화살표 함수가 간결합니다. 반면 객체의 메서드를 정의하거나 `this` 를 사용해야 하는 경우에는 일반 함수(메서드 단축 표기)를 사용하십시오. 화살표 함수는 자신만의 `this` 를 가지지 않기 때문에 메서드 내부에서 `this.속성` 에 접근하면 의도치 않은 값이 됩니다.

Q2. `const` 로 선언한 배열이나 객체도 내용을 바꿀 수 있나요?

A2. 네, 가능합니다. `const` 는 변수 자체의 **재할당**을 막을 뿐, 배열이나 객체 내부의 값 변경은 허용합니다. 예를 들어 `const arr = [1]` 로 선언한 뒤 `arr.push(2)` 는 허용되지만, `arr = [1, 2]` 처럼 새 배열을 할당하는 것은 오류가 납니다.

Q3. `reduce` 의 초기값(`initialValue`)을 언제 꼭 써야 하나요?

A3. 배열이 비어 있을 가능성이 있을 때 반드시 초기값을 지정하십시오. 빈 배열에 초기값 없이 `reduce` 를 호출하면 `TypeError` 가 발생합니다. 또한 누적값의 타입이 배열 요소의 타입과 다를 때(예: 숫자 배열을 객체로 변환하는 경우)에도 초기값이 필요합니다. 습관적으로 초기값을 명시하는 것을 권장합니다.

Q4. `map` 과 `filter` 의 원본 배열이 변경되지 않는다고 했는데, 어떻게 확인하나요?

A4. 다음처럼 직접 확인할 수 있습니다.

```
const original = [1, 2, 3, 4, 5];
const result = original.filter((n) => n > 3);

console.log(original); // [1, 2, 3, 4, 5] - 그대로
console.log(result);   // [4, 5]
```

`map` 과 `filter` 는 항상 새 배열을 반환하므로 원본은 안전합니다.

Q5. 클로저가 메모리 누수를 일으킬 수 있다고 들었습니다. 어떤 경우인가요?

A5. 클로저가 참조하는 외부 변수는 가비지 컬렉터가 수거하지 않습니다. 이벤트 리스너 안에서 큰 객체를 클로저로 참조하고 리스너를 제거하지 않으면 메모리에 계속 남습니다. 11장에서 이벤트 리스너를 학습할 때 `removeEventListener` 로 정리하는 습관을 함께 익히면 됩니다.

장 요약

이번 장에서는 JavaScript 프로그래밍의 핵심 도구인 함수와 자료구조를 다뤘습니다. 함수 선언문·표현식·화살표 함수를 비교하고, 스코프 체인을 통해 변수가 탐색되는 원리와 클로저가 상태를 캡슐화하는 방식을 살펴보았습니다. 배열의 다양한 조작 메서드와 객체의 구조 분해 할당을 익혔으며, `map·filter·reduce` 를 체이닝하여 데이터를 선언적이고 가독성 있게 처리하는 방법을 학습했습니다.

핵심 키워드

함수(Function), 화살표 함수(Arrow Function), 스코프(Scope), 클로저(Closure), 배열(Array), 객체(Object), 구조 분해 할당(Destructuring), `map`, `filter`, `reduce`, 콜백

다음 장 미리보기

11장에서는 지금까지 배운 함수와 배열·객체 지식을 실제 웹 페이지에 적용합니다. **DOM 조작**과 **이벤트 처리**를 통해 버튼 클릭, 사용자 입력 등 상호작용에 반응하는 동적인 UI를 직접 만들어 봅니다.

✂ 실습 프로젝트 (Hands-on Project)

장바구니 금액 계산기

이 장에서 배운 함수, 배열, 객체, map/filter/reduce 개념을 실무 시나리오에 적용하는 프로젝트입니다.

초급: 장바구니 영수증 출력기

시나리오

온라인 쇼핑몰에서 장바구니에 담긴 상품들의 총 금액을 계산하고 영수증을 출력해야 합니다. 상품 데이터는 객체 배열로 관리하며, 선택된 상품만 골라서 최종 금액을 콘솔에 출력합니다. 실무에서 장바구니 UI를 만들기 전에 먼저 데이터 처리 로직을 순수 함수로 작성하는 연습입니다.

학습 목표

- 배열 메서드 filter, map, reduce를 체이닝하여 데이터 처리 파이프라인 구성
- 객체 배열에서 구조 분해 할당으로 필요한 속성만 추출
- 재사용 가능한 순수 함수 작성 (같은 입력이면 항상 같은 결과)

진행 방법

- project_starter.html 을 열고 TODO 주석을 찾습니다
- 각 TODO를 이 장에서 배운 filter, map, reduce, 구조 분해 할당으로 완성합니다
- 완성 후 브라우저에서 열어 콘솔 탭의 출력 결과를 확인합니다

실행 방법

project_starter.html 파일을 브라우저에서 열고, 개발자 도구(F12) > 콘솔 탭을 확

중급: 클로저 기반 장바구니 상태 관리

시나리오

초급에서 만든 데이터 처리 로직을 발전시켜, 클로저로 장바구니 상태를 캡슐화합니다. 외부에서 직접 장바구니 배열을 수정할 수 없고, 오직 제공된 메서드로만 조작할 수 있습니다. 추가/제거/수량 변경 기능을 갖춘 장바구니 객체를 만들고 화면에 실시간으로 렌더링합니다.

학습 목표

- 클로저로 비공개 상태(private state)를 안전하게 캡슐화
- 반환 객체 패턴(revealing module pattern)으로 공개 API 설계
- map/filter/reduce 체이닝으로 동적 UI에 데이터를 반영

진행 방법

1. project_advanced_starter.html 을 열고 TODO 주석을 찾습니다
2. 클로저와 고차 메서드를 조합하여 TODO 부분을 완성합니다
3. 완성 후 브라우저에서 열어 버튼을 클릭하며 장바구니 동작을 확인합니다

실행 방법

project_advanced_starter.html 파일을 브라우저에서 열고, 개발자 도구(F12) > 콘솔

확장 아이디어

- 수량 변경(증가/감소) 기능을 추가해 보십시오
- 상품 카테고리별로 그룹화하여 출력하는 기능을 reduce로 구현해 보십시오
- 쿠폰 코드를 입력하면 총 금액에서 할인을 적용하는 기능을 클로저로 만들어 보십시오

▶  project_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_complete.html (완성 코드)

▶  project_advanced_starter.html (스타터 코드 - 직접 완성해보세요)

▶  project_advanced_complete.html (완성 코드)